Docker

Faire un container:

1. on crée un fichier ou on mettra notre docker-compose
2. on crée dans ce fichier un document texte nommé docker-compose.yml
3. On l'ouvre avec studiocode et on copier l'image que l'on souhaite
4. on enregistre ctrl+s
5. on ouvre le cmd → clique droit → ouvrir dans le terminal
6. on tape docker compose up -d ou docker compose -f nom du docker compose up -d si le docker compose a un nom autre que docker-compose
7. ça va télécharger les fichiers nécessaires
8. Une fois terminé, le nouveau container sera visible dans docker desktop
9. lancer le container
10. Si tout fonctionne correctement il suffira d'aller sur localhost(nom que vous avez donner à l'application) → on peut voir dans la colonne port (une fois l'application lancée, un lien vers le l'app. ex: http://localhost:80)

Commande docker:

- Lancer un docker compose : docker compose up -d ou docker compose -f nom du docker compose up -d
- pour supprimer le container et les fichiers : docker compose -f "nom du docker compose" down -v

Dockerfile:

- Dans le dockerfile, modifier le restart: always en restart: unless-stopped pour que le container ne se lance pas au lancement de docker

Lier une app d'un container à une base de données qui et dans un autre container:

- lier les deux au même network (le mettre en en bas des docker composes):

→ Pour la base de donnée:
ex:

```
networks:
  network_postgres1: #nom du network
    name: postgres1 # nom afficher sur docker
    driver_opts:    # pass options to driver for network creation
      com.docker.network.bridge.host_binding_ipv4: 127.0.0.1
```

→ sur le docker compose de l'application (une fois la base de données créée) ajouter:
"external: true". Cela permet de se connecter à la base de donnée, sinon cela on crée la base de donnée.
ex:

```
networks:
  network_postgres1:
    name: postgres1 # Use a custom name
    external: true
    driver_opts:    # pass options to driver for network creation
      com.docker.network.bridge.host_binding_ipv4: 127.0.0.1
```

Pour finir, il faudra configurer l'environnement pour que ce soit cohérant avec la base de donnée. User , passworduser, host, port, database
Attention:
Ne pas oublier de rajouter :

```
    networks:
      - network_postgres1
```

sous le volume.

Exemple de n8n et postgres:

postgres:

```
services:
  postgresdb:
```

```
    container_name: postgresdb
    image: postgres:13
    restart: unless-stopped
    environment:
      POSTGRES_PASSWORD: 'Password1'
      POSTGRES_DB: 'postgres_production'
      POSTGRES_USER: 'postgresuser'
    volumes:
      - ./postgresdb:/var/lib/postgresql/data
    networks:
      - network_postgres1
    ports:
     - "5432:5432"
  adminer:
    image: adminer
    ports:
      - 8081:8080
    networks:
      - network_postgres1
volumes:
  postgresdb: {}
networks:
  network_postgres1:
    name: postgres1 # Use a custom name
    driver_opts:    # pass options to driver for network creation
      com.docker.network.bridge.host_binding_ipv4: 127.0.0.1
```

n8n:

Database adress:

Exemple d'adresse:
DATABASE_URL:postgresql://postgresuser:Password1@postgresdb:5432/baserowdb

Élément	Valeur	Description
Protocole / SGBD	`postgresql`	Indique qu'on utilise une base de données PostgreSQL.
Utilisateur	`postgresuser`	Le nom d'utilisateur utilisé pour se connecter à la base.
Mot de passe	`Password1`	Le mot de passe associé à l'utilisateur.
Hôte (serveur)	`postgresdb`	Le nom d'hôte ou le conteneur (dans Docker, souvent un service nommé `postgresdb`).
Port	`5432`	Le port par défaut de PostgreSQL.
Nom de la base de données	`baserowdb`	Le nom de la base à laquelle on se connecte.

🔹 En résumé

Cette ligne signifie :

> "Connecte-toi à la base baserowdb sur le serveur postgresdb (port 5432) avec l'utilisateur postgresuser et le mot de passe Password1, en utilisant PostgreSQL."

🧩 1. Dans un fichier `.env`

Tu peux la placer telle quelle :

```
DATABASE_URL=postgresql://postgresuser:Password1@postgresdb:5432/baserowdb
```

👉 Ce fichier `.env` sert à stocker les variables d'environnement sans les exposer dans le code du docker compose.

Créer un network avec une base de donnée:

```
    networks:
      macvlan_network:
        ipv4_address: 10.2.0.175
```

```
networks:
  macvlan_network:
    driver: macvlan
    driver_opts:
      parent: eth0
    ipam:
      config:
        - subnet: 10.2.0.0/24
          gateway: 10.2.0.254
```

lier une app a la base de donnée:

```
    networks:
      mysql-vlan_macvlan_network:
        ipv4_address: 10.2.0.162
networks:
  mysql-vlan_macvlan_network:
    external: 'true'
```